

Student Expense Tracker

High-Level Design Document

Your Name
Your College / University

September 28, 2026

Abstract

This document provides a high-level architectural overview of the Student Expense Tracker, an AI-assisted web application that helps students record their expenses, manage monthly budgets, and understand their spending habits. The system combines simple expense recording, budget tracking, synchronized multi-device data, and AI-generated spending insights with a clean, student-focused user interface. This document focuses on system architecture, component interactions, and major design decisions without implementation-level code details.

Contents

1	System Overview	3
1.1	Platform Vision	3
1.2	Core Capabilities	3
1.3	Technology Philosophy	3
2	Architectural Blueprint	4
2.1	High-Level System Architecture	4
2.1.1	Architectural Components	4
2.1.2	Communication Patterns	4
2.2	Data Flow Architecture	5
2.2.1	Key Data Pathways	5
2.2.2	Consistency Model	5
3	Component Architecture	7
3.1	Frontend Component Structure	7
3.1.1	Architectural Layers	7
3.2	UI/UX Design Strategy	8
3.2.1	Design System Principles	8
3.2.2	User Experience Patterns	8
4	Backend Architecture	9
4.1	Service Architecture	9
4.1.1	Service Components	9
4.2	Data Model Design	10
4.2.1	Core Entities	10
4.2.2	Relationship Patterns	10

5	Real-time Communication Architecture	11
5.1	Supabase Realtime Implementation Strategy	11
5.1.1	Communication Patterns	11
5.1.2	Event Management	12
5.2	Consistency and Reliability	12
6	Security Architecture	13
6.1	Authentication and Authorization	13
6.1.1	Authentication Strategy	13
6.1.2	Authorization Model	13
6.2	Security Measures	13
7	Key Feature Workflows	15
7.1	Expense Recording and Budget Update	15
7.1.1	User Interaction Flow	15
7.1.2	Technical Implementation Approach	15
7.2	AI Spending Insights	17
7.2.1	Insight Generation Processing	17
7.2.2	AI Integration Strategy	17
8	System Integration	18
8.1	External Service Integration	18
8.2	API Design Principles	18
8.3	Deployment Architecture	18
9	Performance and Scalability	18
9.1	Performance Characteristics	18
9.2	Scalability Strategy	18
10	Conclusion	19
10.1	Architectural Strengths	19
10.2	Implementation Readiness	19
10.3	Future Considerations	19

1 System Overview

1.1 Platform Vision

The Student Expense Tracker is designed as a modern, AI-enhanced personal finance tool for students, enabling them to understand where their money goes through intuitive interfaces and intelligent insights. The system bridges traditional expense-tracking methods with modern technologies including managed cloud backends, secure per-user data isolation, and AI-driven analysis. It is a *tracker*, not a planner: it records what the student actually spent and reports on it.

1.2 Core Capabilities

- **Fast Expense Recording:** Add, edit, and delete income and expense entries in a few taps
- **Budget Tracking:** Overall and category-wise monthly budgets with usage alerts
- **Spending Analytics:** Visual daily, weekly, and monthly spending charts
- **AI Spending Insights:** Simple, plain-language observations generated from summarised spending data
- **Multi-device Sync:** Changes appear across the student's devices without a page refresh
- **Data Portability:** CSV export of transactions

1.3 Technology Philosophy

The system adopts a modern JavaScript front-end with a managed backend-as-a-service, with emphasis on:

- **Developer Experience:** Modern tooling and clear separation of concerns
- **User Experience:** Responsive, minimal design that works well on a phone
- **Scalability:** Modular architecture supporting future expansion
- **Performance:** Optimized loading and responsive interactions

2 Architectural Blueprint

2.1 High-Level System Architecture

The system follows a three-tier architecture with clear separation between presentation, business logic, and data layers, enhanced by real-time synchronization capabilities.

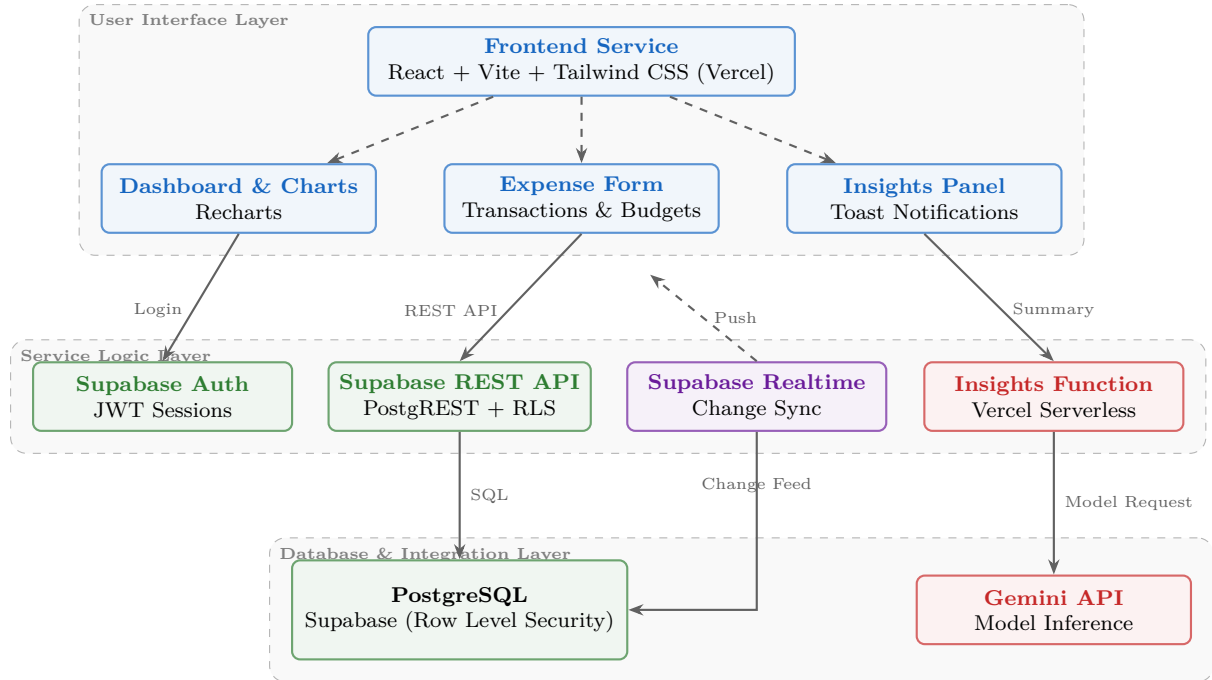


Figure 1: System Architecture Overview

2.1.1 Architectural Components

1. **Client Layer (Frontend):** React-based single-page application with responsive design
2. **Application Layer (Backend):** Supabase-managed API with authentication and business rules enforced through database policies
3. **Real-time Layer:** Supabase Realtime for change notifications across devices
4. **AI Service Layer:** Isolated serverless function that calls the Gemini API
5. **Data Layer:** Supabase PostgreSQL for primary data storage
6. **External Services:** Gemini API for insight generation

2.1.2 Communication Patterns

- **RESTful API:** For structured data operations (transactions, categories, budgets)
- **WebSocket Connections:** For real-time change notifications
- **Service-to-Service:** Serverless function to the AI provider
- **External API Calls:** Integration with the Gemini API

2.2 Data Flow Architecture

The system implements a data flow that ensures consistency while providing a responsive, real-time user experience.

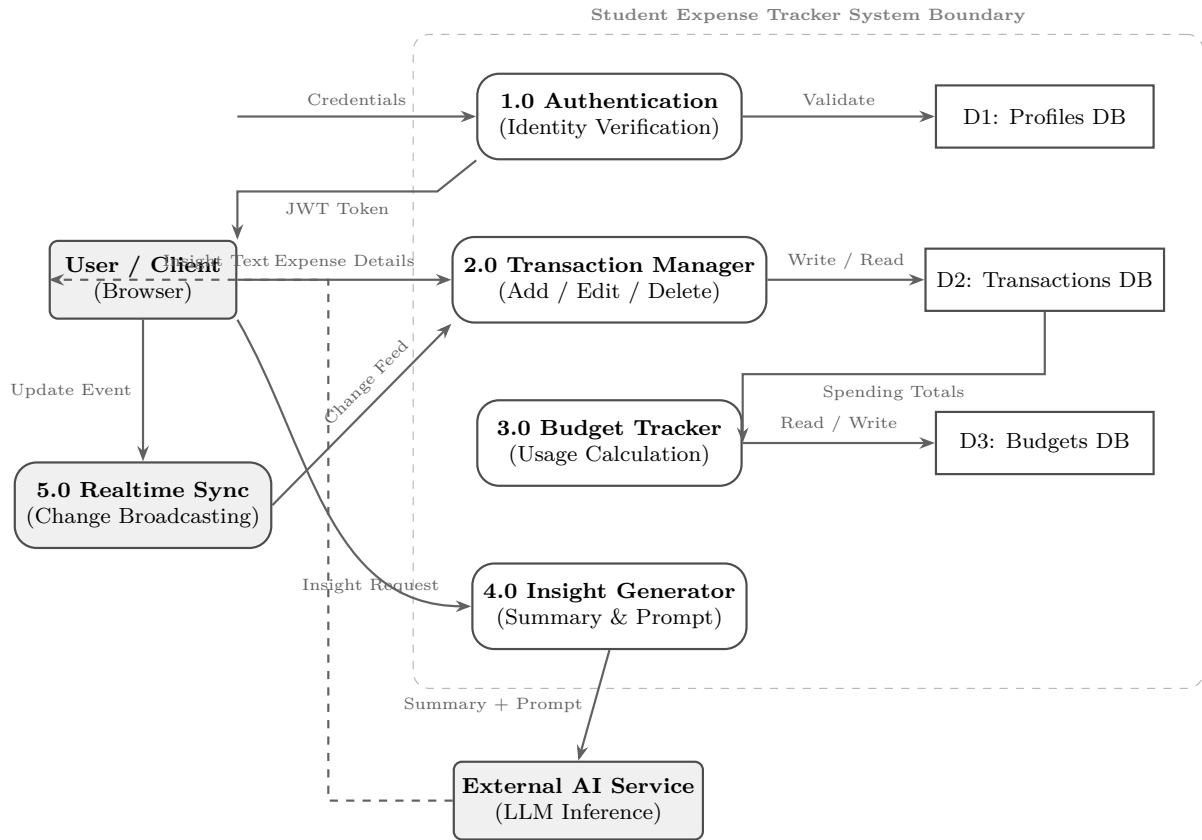


Figure 2: System Data Flow Diagram

2.2.1 Key Data Pathways

- 1. Authentication Flow:** User credentials → JWT token generation → Session establishment
- 2. Transaction Management Flow:** User interaction → Optimistic UI update → Database persistence → Real-time notification
- 3. Budget Tracking Flow:** Transaction change → Usage recalculation → Threshold check → Alert display
- 4. AI Insight Flow:** Summarised statistics → Prompt construction → AI processing → Insight display

2.2.2 Consistency Model

- **Database-First Approach:** All state changes committed to the database before other devices are notified
- **Optimistic Updates:** Immediate UI feedback during user operations
- **Eventual Consistency:** Real-time synchronization across all of a student's devices

- **Conflict Resolution:** Timestamp-based conflict detection and resolution (latest update wins)

3 Component Architecture

3.1 Frontend Component Structure

The frontend follows a modular React architecture with clear separation between infrastructure, state management, and UI components.

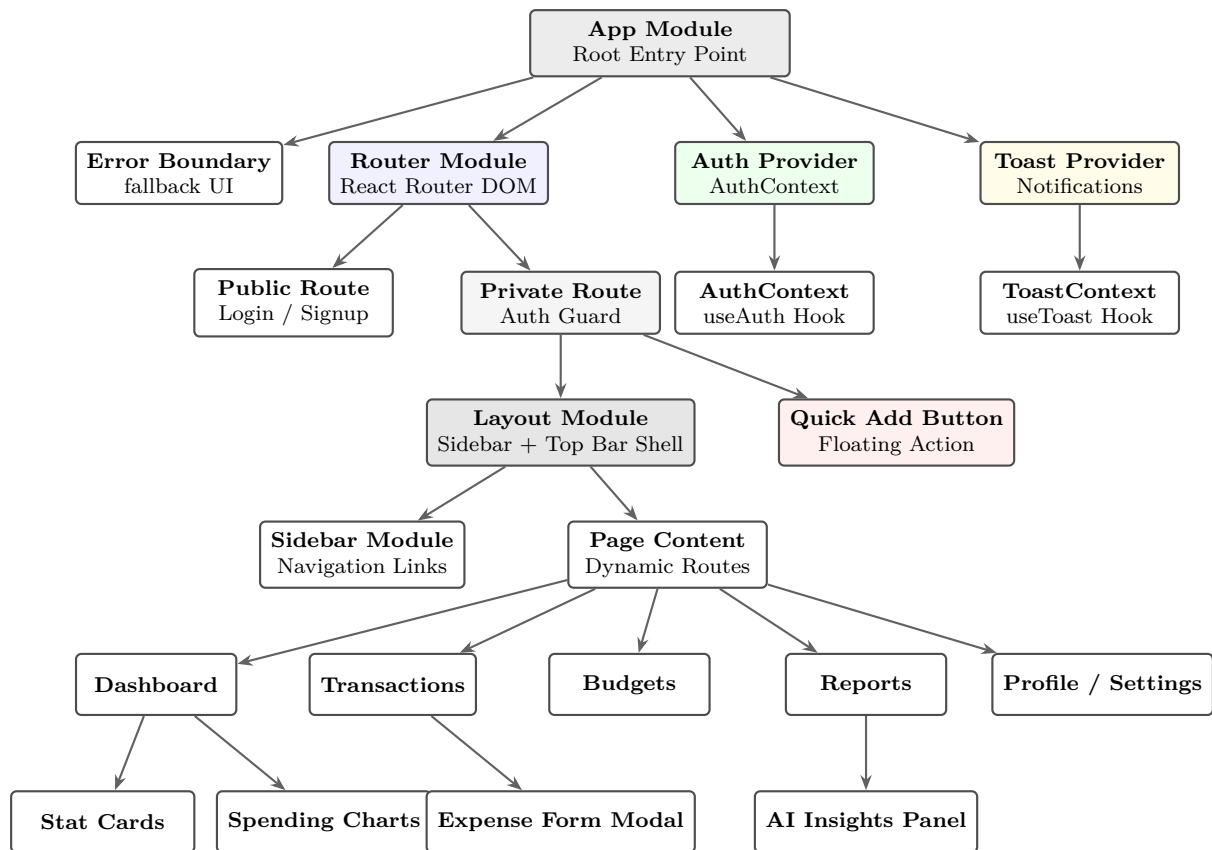


Figure 3: React Component Hierarchy

3.1.1 Architectural Layers

1. **Infrastructure Layer:** Application bootstrap, routing, and error handling

- Application root with error boundary protection
- Routing configuration for public and private routes
- Global context providers for state management

2. **State Management Layer:** Application-wide state coordination

- Authentication context for user session management
- Toast context for success and error notifications
- Custom hooks for clean state consumption

3. **Layout Layer:** Application structure and navigation

- Main layout with sidebar on desktop and mobile menu on small screens
- Consistent top bar and navigation across all pages

- Protected route wrapper for authentication
- 4. **Feature Layer:** Business functionality implementation
 - Dashboard with spending totals and remaining budget
 - Transactions page with add, edit, filter, and delete
 - Budgets page with usage progress bars
 - Reports page with charts and AI insights
- 5. **UI Component Layer:** Reusable presentation components
 - Stat cards and transaction rows
 - Modal dialogs for the expense form
 - Form elements with validation
 - Data visualization widgets

3.2 UI/UX Design Strategy

3.2.1 Design System Principles

- **Clean and Minimal Aesthetic:** Simple visual hierarchy without decorative clutter
- **Light and Dark Themes:** Readable colour schemes with sufficient contrast
- **Subtle Motion:** Restrained transitions for state changes
- **Responsive Adaptation:** Mobile-first design with progressive enhancement

3.2.2 User Experience Patterns

- **Quick Entry:** Add an expense in a few taps from any page
- **Contextual Actions:** Edit and delete controls shown on each transaction row
- **Progressive Disclosure:** Filters and advanced settings revealed as needed
- **Multi-modal Interaction:** Support for mouse, touch, and keyboard input

4 Backend Architecture

4.1 Service Architecture

The backend follows a modular service-oriented architecture built on a managed platform, with clear separation of concerns and well-defined interfaces.

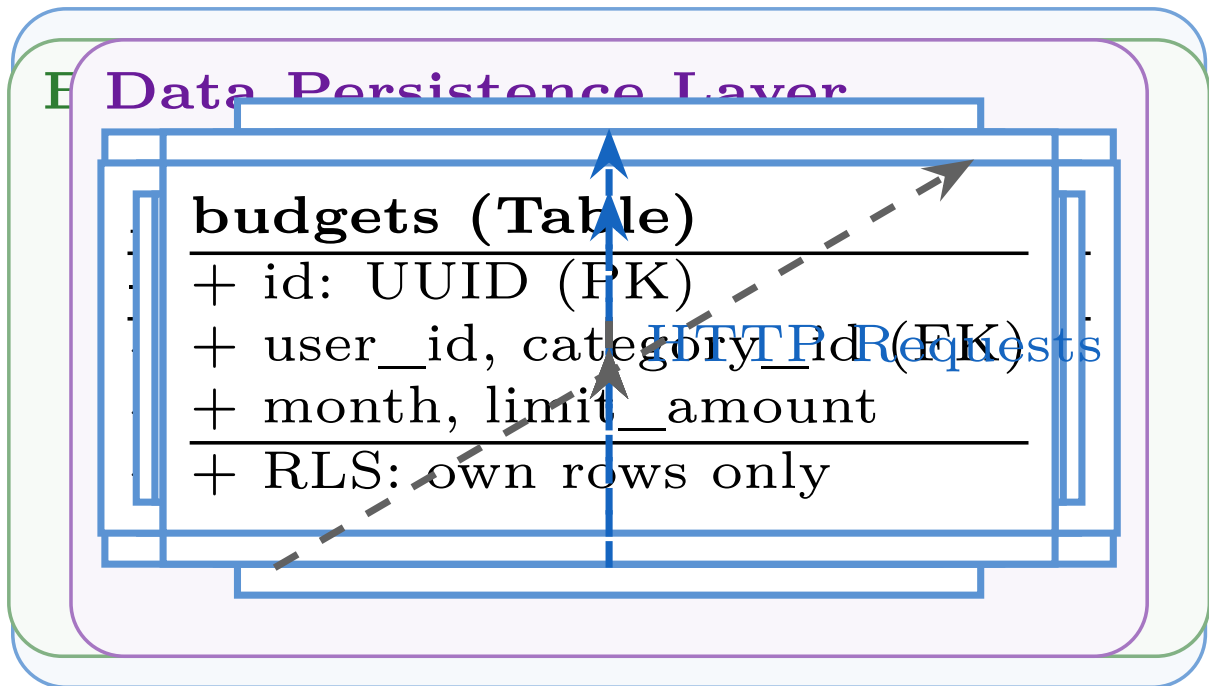


Figure 4: Backend Class Structure

4.1.1 Service Components

1. **Authentication Service:** User identity management and session handling
 - JWT token generation and validation
 - Password security and encryption
 - Session persistence and management
2. **Transaction Management Service:** Core expense operations
 - Transaction creation, editing, and deletion
 - Category assignment and custom categories
 - Filtering by date range, category, type, and payment method
3. **Budget Management Service:** Budget operations and evaluation
 - Overall and category-wise monthly budgets
 - Budget usage calculation from recorded expenses
 - Threshold alerts at configurable levels (default 80% and 100%)
4. **AI Processing Service:** Intelligent insight functionality
 - Prompt construction from summarised statistics

- Insight generation through the Gemini API
- Graceful fallback when the AI service is unavailable

5. **Real-time Service:** Change notification management

- Subscription lifecycle management
- User-scoped change broadcasting
- Reconnection handling

4.2 Data Model Design

4.2.1 Core Entities

- **Profile:** System users with name, preferred currency, and preferences
- **Category:** Default and custom spending categories
- **Transaction:** Income or expense records with amount, date, payment method, and note
- **Budget:** Monthly spending limits, overall or per category
- **Insight Feedback:** User responses to dismissed AI insights

4.2.2 Relationship Patterns

- **One-to-Many:** User owns multiple categories, transactions, and budgets
- **Many-to-One:** Each transaction belongs to one category
- **Optional Reference:** A budget with no category is the overall monthly budget
- **Row Ownership:** Every row carries the owning user, enforced by row-level policies

5 Real-time Communication Architecture

5.1 Supabase Realtime Implementation Strategy

The system employs Supabase Realtime for change notifications with per-user scoping so that a student's devices stay synchronized.

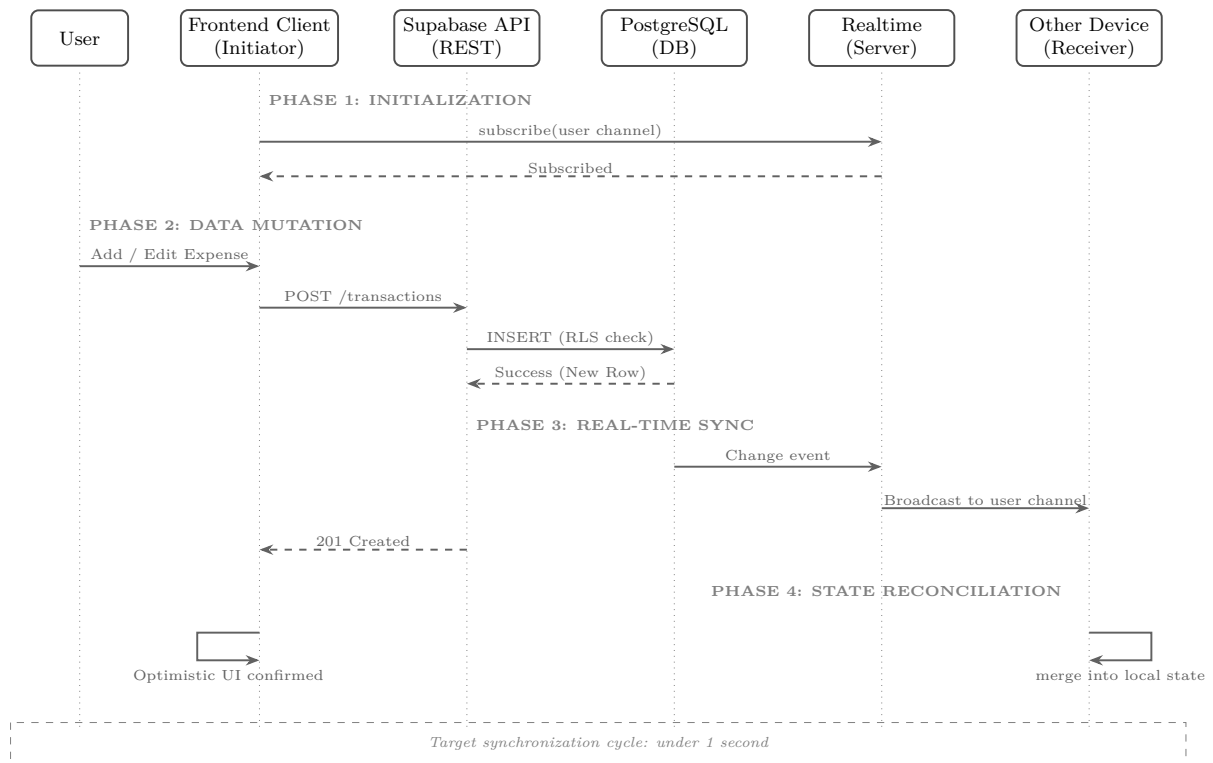


Figure 5: Real-Time Communication Architecture

5.1.1 Communication Patterns

1. **User-Scoped Broadcasting:** Changes sent only to the owning user's devices
 - Transaction changes pushed to the user's channel
 - Budget updates reflected on all open sessions
 - Category changes synchronized across devices
2. **Table-Level Subscriptions:** Change feeds per data table
 - Insert, update, and delete events
 - Filtered by the authenticated user
 - Dashboard totals refreshed on relevant events
3. **Local Notifications:** In-app messages triggered by events
 - Budget threshold warnings
 - Success and error toasts
 - Weekly insight availability

5.1.2 Event Management

- **Event Classification:** Categorization by table and operation type
- **Guaranteed Delivery:** Refetch of latest data after reconnection
- **Event History:** Latest state fetched from the database on resume
- **Rate Limiting:** Protection against event flooding through platform limits

5.2 Consistency and Reliability

- **Database-First Consistency:** All changes persisted before events are emitted
- **Connection Recovery:** Automatic reconnection with state synchronization
- **Event Ordering:** Timestamps used for dependent operations
- **Conflict Prevention:** Last-write-wins policy with server timestamps

6 Security Architecture

6.1 Authentication and Authorization

The system implements a comprehensive security model with multi-layered protection mechanisms.

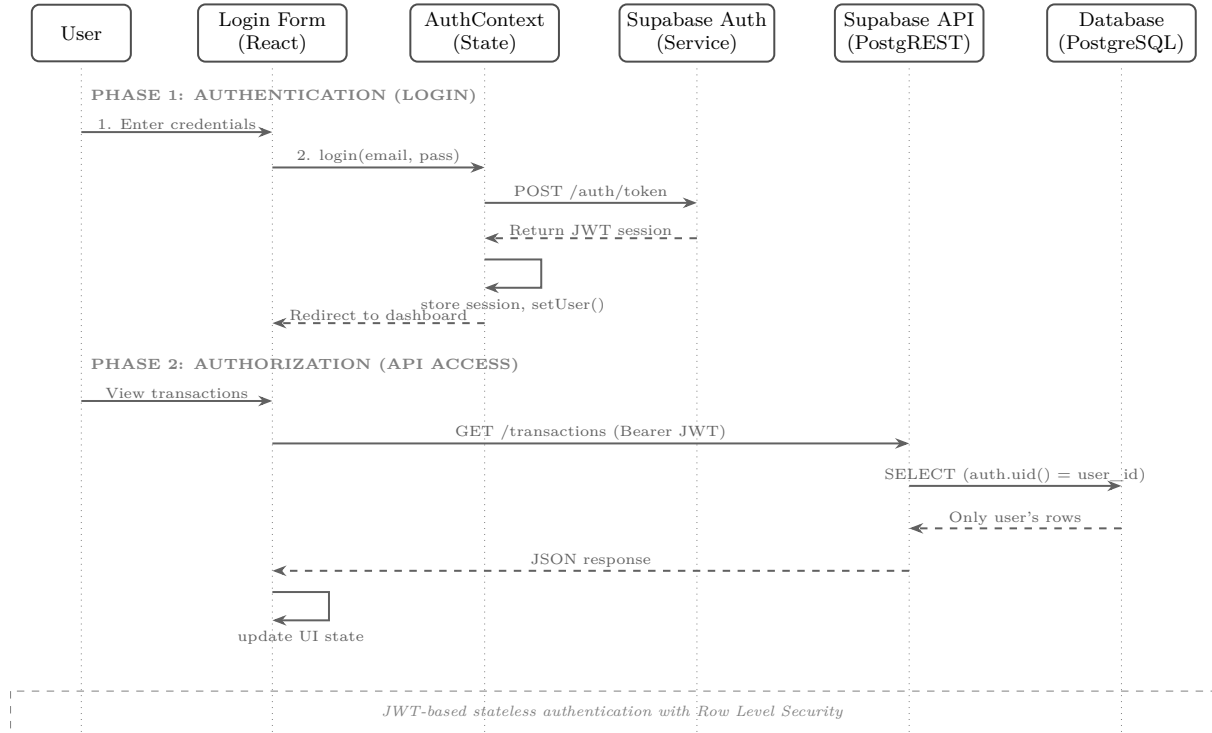


Figure 6: Authentication and Authorization Flow

6.1.1 Authentication Strategy

1. **Credential-Based Authentication:** Email/password with secure hashing
2. **Token-Based Sessions:** JWT tokens for stateless authentication
3. **Session Management:** Token refresh and sign-out mechanisms
4. **Social Sign-in Options:** Google sign-in through OAuth 2.0

6.1.2 Authorization Model

- **Row Level Security (RLS):** Database policies restrict every table to the owning user
- **Resource Ownership:** Users can read and modify only their own records
- **Role Separation:** Public (anon) key with limited rights in the client, privileged keys never exposed
- **Future Admin Role:** Separate role for default category management

6.2 Security Measures

- **Data Encryption:** HTTPS enforcement with strong cipher suites

- **Input Validation:** Comprehensive sanitization of all user inputs
- **Secret Management:** AI provider key kept in server-side environment variables only
- **Content Security:** CSP headers and XSS protection mechanisms
- **Privacy by Design:** Only summarised statistics sent to the AI service

7 Key Feature Workflows

7.1 Expense Recording and Budget Update

The expense form implements a fast, validated entry flow with instant feedback and automatic budget recalculation.

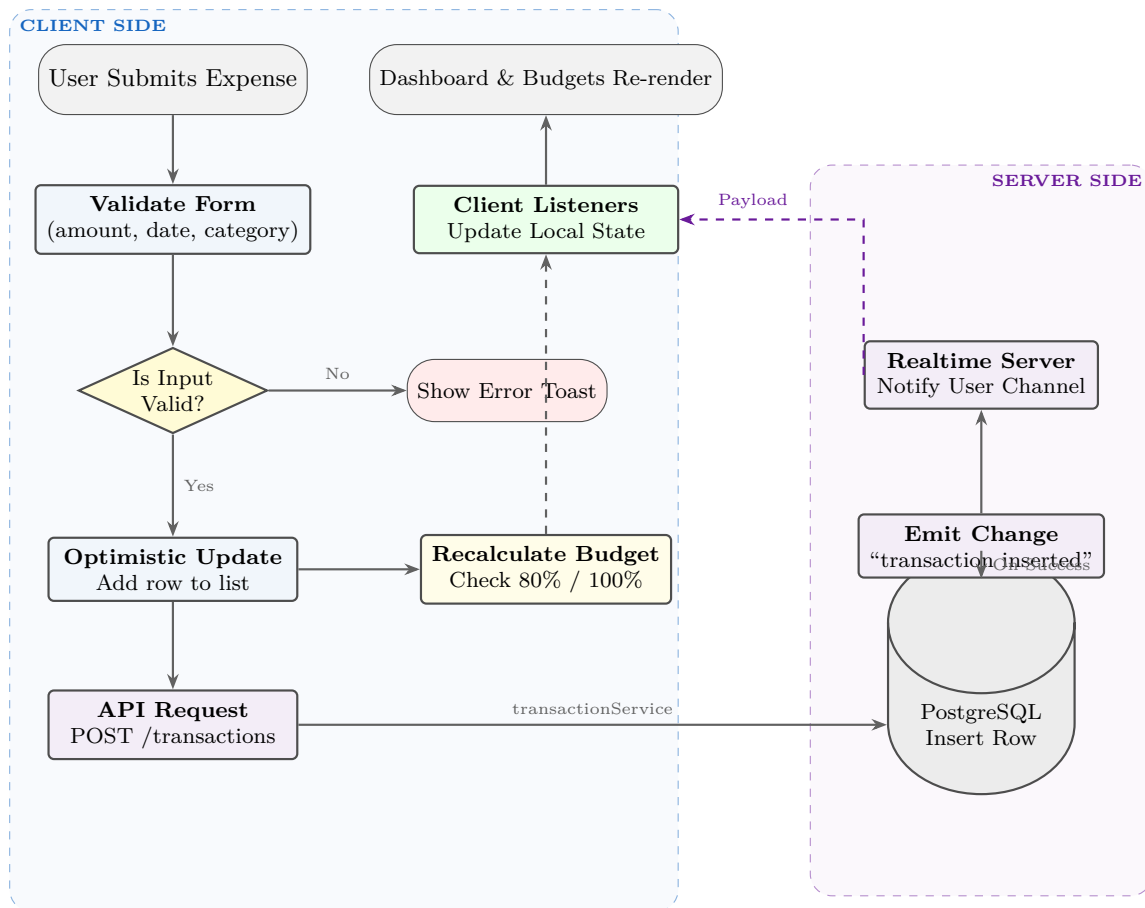


Figure 7: Expense Recording Workflow

7.1.1 User Interaction Flow

1. **Entry Initiation:** User opens the expense form from the dashboard or quick-add button
2. **Field Entry:** Amount, category, date, payment method, and optional note are entered
3. **Validation:** Amount must be positive and date valid before submission
4. **Instant Feedback:** Transaction appears in the list immediately
5. **Budget Recalculation:** Budget usage and remaining amount updated
6. **State Synchronization:** Changes propagated to the user's other devices

7.1.2 Technical Implementation Approach

- **Optimistic Updates:** Immediate UI response before server confirmation, with rollback on failure

- **Derived Statistics:** Totals and budget usage computed from stored transactions rather than hard-coded
- **Error Handling:** Friendly toast messages instead of technical errors
- **Performance Optimization:** Indexed queries on user and date to minimize database load

7.2 AI Spending Insights

The insights feature turns summarised spending statistics into short, plain-language observations and never blocks the rest of the application.

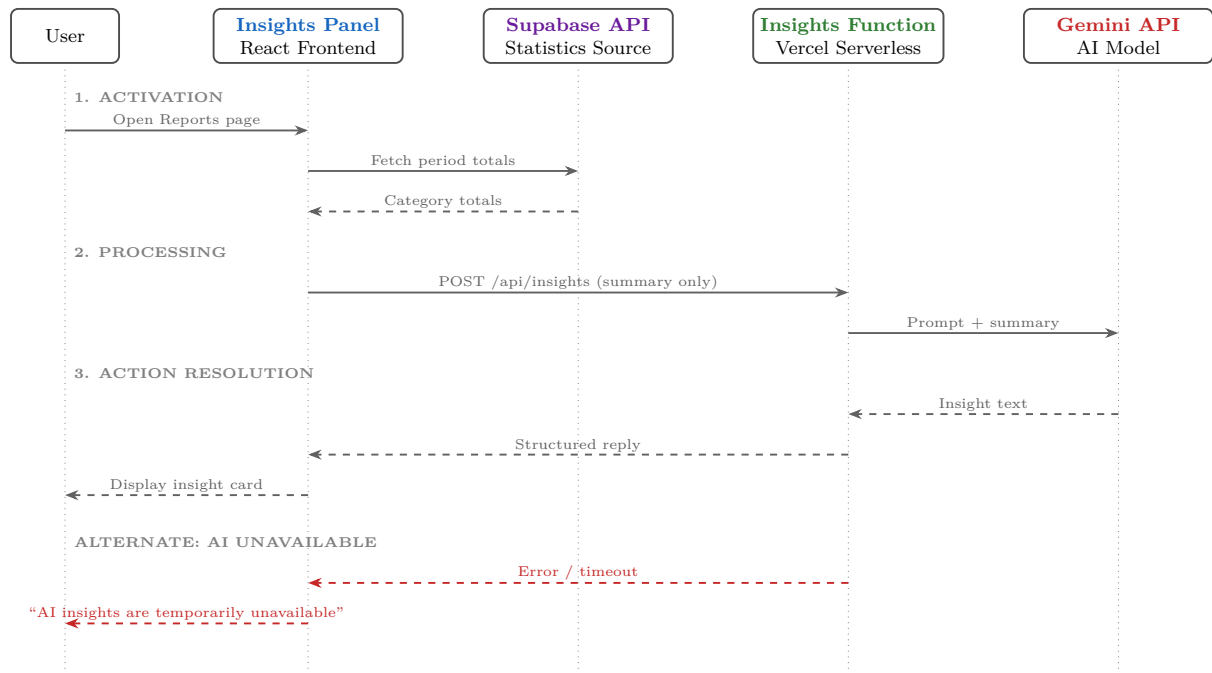


Figure 8: AI Insight Generation Workflow

7.2.1 Insight Generation Processing

1. **Data Aggregation:** Period totals and category totals computed from stored transactions
2. **Privacy Filtering:** Only summarised statistics are prepared; notes and personal details are excluded
3. **Prompt Construction:** Summary formatted into a short, structured prompt
4. **AI Processing:** The serverless function requests an insight from the Gemini API
5. **Response Delivery:** Insight text returned and displayed in the insights panel
6. **Feedback Capture:** Dismissed insights record a reason to avoid repeats

7.2.2 AI Integration Strategy

- **Summary Understanding:** Highest-spending categories and week-on-week changes
- **Budget Awareness:** Suggestions when a budget is close to being exceeded
- **Action Generation:** Simple, actionable tips rather than complex plans
- **Learning Adaptation:** Reduced repetition based on dismissal feedback
- **Graceful Degradation:** Timeouts and errors show a friendly message while statistics remain available

8 System Integration

8.1 External Service Integration

- **Backend Platform:** Supabase for authentication, database, and real-time updates
- **AI Services:** Gemini API for enhanced spending insights
- **Hosting Platform:** Vercel for static hosting and serverless functions
- **Data Export:** CSV export of transactions for use in spreadsheets

8.2 API Design Principles

- **RESTful Design:** Resource-oriented API with consistent patterns
- **Version Management:** Backward-compatible database and API changes
- **Documentation:** Clear project documentation with request examples
- **Rate Limiting:** Fair usage limits on the AI endpoint

8.3 Deployment Architecture

- **Static Hosting:** Vite production build served from Vercel's edge network
- **Serverless Functions:** AI insight endpoint deployed alongside the frontend
- **Managed Database:** Supabase-hosted PostgreSQL with automated backups
- **Environment Configuration:** Keys and URLs stored as environment variables

9 Performance and Scalability

9.1 Performance Characteristics

- **Initial Load Time:** Optimized bundle delivery with code splitting
- **Real-time Responsiveness:** Target of sub-second update propagation
- **Concurrent User Support:** Managed platform scaling for user load
- **Data Throughput:** Efficient data transfer with compression

9.2 Scalability Strategy

- **Horizontal Scaling:** Stateless frontend and serverless functions supporting multiple instances
- **Database Indexing:** Indexes on user and date for large transaction histories
- **Caching Layers:** Browser caching and cached aggregate queries for frequently accessed data
- **Load Distribution:** Edge network routing provided by the hosting platform

10 Conclusion

10.1 Architectural Strengths

- **Modern Technology Stack:** Leverages current best practices and tools
- **Real-time Capabilities:** Seamless multi-device synchronization
- **AI Integration:** Intelligent assistance that never blocks core features
- **Modular Design:** Clear separation facilitating maintenance and expansion
- **Security Focus:** Row-level protection and secret management throughout

10.2 Implementation Readiness

The system architecture represents a well-designed foundation ready for implementation. The modular approach allows for parallel development, while the clear separation of concerns ensures maintainability. The use of a managed backend provides both current functionality and future expansion capabilities.

10.3 Future Considerations

- **Mobile Applications:** Native mobile app development for iOS and Android
- **Offline Functionality:** Progressive web app capabilities for limited connectivity
- **Advanced Analytics:** Predictive insights and forecasting
- **Extended Integrations:** Receipt scanning and bank statement import
- **Internationalization:** Multi-language and multi-currency support

The Student Expense Tracker architecture provides a robust foundation for building a simple, useful money-tracking application that combines traditional expense recording with modern cloud services and AI assistance.